Mobile App Development

Task Title:	**Shared Living App Challenge (Expenses, Shopping & Budget)**
Duration:	**5 HOURS**
Total Marks:	100 MARKS
Team Configuration:	2 to 3 Members
Architecture Scope:	**Full-Stack (Mobile Frontend App + Backend REST API)**

General Instructions

- This task is not bound to any specific language either on a front-end or back-end (use any suitable technology stack for the mobile applications).
- Detailed Evaluation Rubrics of Mobile App Development are mentioned below.
- The application must be developed using your own logic, design, and implementation approach
- Try to write clean code which can be easily understandable.
- Try to schedule your activities w.r.t to your specified time limit (Late
- Submission will not be evaluated)
- Internet access is now permitted for documentation lookup, dependency Installation, and general research.
- However, use of any AI-assisted coding tools (such as Cursor, Lovable, GitHub Copilot, ChatGPT Code Generator, etc.) is strictly prohibited.
- AI-generated or AI-assisted code will result in disqualification
- Technical Malpractice are not allowed (Unapproved use of devices, software, or unauthorized tools).

---

1. The Everyday Problem Scenario

Ali, Ayesha, and Bilal share an apartment in Lahore. One buys groceries on the way home, another pays the internet bill, and receipts routinely get lost in messaging group chats. Purchases are sometimes logged twice, and no one is ever sure who owes money to whom at the end of the month.

Your challenge is to build a full-stack mobile application (Mobile Client + Backend REST API) that serves as a single, dependable platform to manage shopping items, log shared expenses, split bills accurately, settle debts, and track monthly household budgets.

Detailed Problem Specifications

Question 1: Backend Architecture & REST APIs (25 Marks)

Build a lightweight Backend REST API server (using Node.js/Express, Python FastAPI/Flask, Django, C#/.NET, Java Spring Boot, Go, or Firebase/Supabase REST endpoints) supported by a persistent database (SQLite, PostgreSQL, MongoDB, or JSON storage).

Your backend MUST expose the following RESTful API endpoints:

Method & Endpoint	Purpose / Description	Key Payload / Output
GET /api/members	Fetch household members list	Returns array of members [{id, name}]
GET /api/shopping POST /api/shopping	Get shopping items or add new item	Item object: {id, title, unit, requested_qty, status}
POST /api/shopping/checkout	Checkout selected items & auto-post expense	Payload: {item_ids, total_paid, payer_id, split_members}
POST /api/expenses	Create manual shared expense	Payload: {title, category, amount, payer_id, split_type, splits}
GET /api/balances	Get current net balance per member	Returns member net balances (sum must = 0.00)
POST /api/settle	Record debt payment between 2 members	Payload: {sender_id, recipient_id, amount}

Question 2: Mobile Frontend & User Experience (25 Marks)

Develop a clean mobile client (Flutter, React Native, Native Android/Kotlin, or iOS/Swift) providing coherent navigation (Tab Bar or Drawer Navigation) across the following screens:

1. Home Dashboard:
- Current month total spending & monthly budget progress bar.
- Active member selector (allows switching between Ali, Ayesha, and Bilal).
- Net balance card for the active member ("You are owed PKR 250.00" or "You owe PKR 450.00").
- Count of remaining pending shopping items.

2. Shopping List & Checkout Screen:
- List of items with status tags (Pending, Bought).

- Form to add item (title, unit: pack, dozen, bottle, bag, requested quantity).
- Multi-select checkout flow: Select items, enter total price paid, choose payer, and submit checkout via API.

3. Expenses & Settle Up Screen:
- Log manual shared expense (title, category, date, amount, payer, split strategy).
- Display live member balance summary table.
- 'Settle Up' dialog: Select sender, recipient, and payment amount to record cash settlements.

Question 3: Financial Logic, Fair Splitting & Settlement Engine (20 Marks)

Implement exact monetary splitting algorithms in PKR with two decimal place precision:

• **Equal Split Strategy**: Distribute amount evenly across selected members. If the split leaves leftover paisa (cents), distribute remainder paisa 1-by-1 in ascending member ID order (M01, M02, M03) to ensure total exact sum conservation.
• **Exact Rounding Check Rule:** For a PKR 1,000.00 expense shared equally among M01 Ali, M02 Ayesha, and M03 Bilal, the exact allocated shares MUST be M01: 333.34, M02: 333.33, M03: 333.33.
• **Net Balance Equation**: For any member i:

Net Balance = Total Amount Paid - Total Share Owed + Settlements Received - Settlements Sent

The sum of all net balances across the household MUST always equal exactly PKR 0.00.
• **Settlement Validation Rules**: Settlements transfer money directly from a debtor to a creditor without increasing total household spending. Reject non-positive amounts, self-payments, and over-settlements (amount exceeding debtor's debt or recipient's credit).

Question 4: Full-Stack Integration & Error Handling (10 Marks)

Ensure robust HTTP client communication between the Mobile Client and Backend REST API:
• Provide loading spinners / indicators during asynchronous API requests.
• Prevent double submission (disable submit buttons while checkout/expense API calls are processing).
• Display clear user-friendly error banners/toasts when backend validation fails or network connectivity drops.
• Persistent Database: App data must persist across mobile app restarts and backend API server restarts.

Question 5: Live Surprise Requirement (Released at Hour 3.5) (10 Marks)

At Hour 3.5 (1.5 hours before deadline), the organizers will release a new product requirement modifying an existing workflow. Teams will have 1 hour to integrate this new feature into both backend API and mobile UI while keeping existing features intact. See Annexure A on the following page for official organizer surprise task choices and release guidelines.

Question 6: Code Walkthrough & Demonstration (10 Marks)

Submit the project at the 5-hour deadline. Each team will present a 5-minute live demonstration to judges:

1. Complete a live shopping checkout trip, log 2 shared bills, view updated balances, record a settlement, and check budget status.
2. Oral Code Defense (6 Marks): Each team member must individually walk through and explain specific portions of the codebase (e.g., API routes, state management, rounding algorithms). Failure to explain code will lead to severe mark deductions under the No-AI rule.

4. Standard Demonstration Test Data

Use the following official benchmark test case to verify your financial and API logic during development and judging:

Item / Expense Title	Paid By	Amount (PKR)	Split Details	Resulting Household Spending
Grocery Trip (Milk & Rice)	Ali (M01)	780.00	Equal across Ali, Ayesha, Bilal (260.00 each)	780.00
Cleaning Supplies	Ayesha (M02)	900.00	Equal between Ali & Ayesha (450.00 each)	1,680.00
Internet Bill	Bilal (M03)	1,500.00	Custom: Ali 500, Ayesha 300, Bilal 700	3,180.00
Electricity Bill	Ali (M01)	1,020.00	Equal across Ali, Ayesha, Bilal (340.00 each)	4,200.00

Expected Benchmark Financial Summary (Before Settlements):

• Total Household Spending: PKR 4,200.00

• Household Monthly Budget: PKR 5,000.00 | Remaining Budget: PKR 800.00 (84% used - Status: 'Near Limit')

• **Net Balances Owed / Receivable**:

- Ali (M01): Paid 1,800.00 | Share Owed 1,550.00 | Net Balance = +250.00 (Should RECEIVE PKR 250.00)
- Ayesha (M02): Paid 900.00 | Share Owed 1,350.00 | Net Balance = -450.00 (OWES PKR 450.00)
- Bilal (M03): Paid 1,500.00 | Share Owed 1,300.00 | Net Balance = +200.00 (Should RECEIVE PKR 200.00)

• **Settlement Execution:**

1. Ayesha pays Ali PKR 250.00 (Updates Ali balance to 0.00, Ayesha balance to -200.00).
2. Ayesha pays Bilal PKR 200.00 (Updates Bilal balance to 0.00, Ayesha balance to 0.00).
3. Post-Settlement Spending remains exactly PKR 4,200.00 (Settlements do not alter spending).

Evaluation Rubrics & Marks Distribution (100 Marks)

The submission will be evaluated based on the standard 4-pillar mobile application development rubrics below. Each pillar maps directly to specific functional deliverables in the Shared Living App Challenge:

Rubric Criterion	Marks	Evaluation Description	Application Task Mapping
Functionality	**50**	Does the system cover all maximum functional requirements accurately? - Backend REST API endpoints & database persistence - Shopping list & checkout expense posting - Shared expense splitting engine & debt settlements - Monthly budget tracking & status - Live Surprise Task integration (Hour 3.5)	• Task 1.1: REST API (15M) • Task 1.2: Shopping Checkout (10M) • Task 1.3: Expense & Split Logic (15M) • Task 1.4: Hour 3.5 Surprise (10M)
Design	**20**	Is the interface clean, modern, responsive, and user-friendly? - Clear information hierarchy & visual layout - Accessible UI components & consistent spacing - Seamless navigation (Tab bar / Drawer, screens, modals) - Clear monetary labels (PKR) & status indicators	• Dashboard Screen • Shopping List & Checkout UI • Expense Entry & Settlement Dialog • Navigation Structure
Code Quality	**10**	Is the code well-structured and maintainable? - Adherence to standard development practices - Logical structure & clear naming conventions - Separation of concerns (API routes vs logic vs UI) - Sensible state management & component reusability	• Backend Controller / Route modularization • Frontend State Management • Clean API Client implementation

User Experience & Verification	**20**	Is the overall experience smooth, resilient, and verifiable? - Validation & feedback (preventing over-settlements & duplicate submit) - Basic resilience (loading indicators & network error handling) - Code Walkthrough & Defense (6 Marks) verifying genuine implementation without AI	• Error toasts & loading spinners • Prevention of double-taps • Individual Oral Code Defense (6 Marks) • Live Demonstration (4 Marks)

ANNEXURE A: HOUR 3.5 SURPRISE TASK OPTIONS

Official Challenge Released at Hour 3.5 (10 Marks | Time Allowed: 1 Hour)

Option 1 (Recommended): Minimal Debt Settlement Optimizer ('Smart Settle')

Scenario:

In a busy household, multiple people owe money to multiple roommates. Instead of making 4 or 5 separate pairwise payments, the roommates want the app to calculate the absolute minimum number of payments needed to settle all household debts completely.

Deliverables & Requirements:

1. Backend REST API Endpoint (GET /api/balances/simplify):

- Create an endpoint that processes all current member net balances and calculates the minimal set of transfer payments required.
- Algorithm: Pair the largest debtor with the largest creditor to resolve balances greedily until all debts reach PKR 0.00.

2. Mobile UI Implementation:

- Add a 'Smart Settle Suggestions' button on the Balances Screen.
- Clicking the button fetches the optimized transfer list and displays simple card recommendations (e.g., 'Ayesha pays Ali PKR 250.00' and 'Ayesha pays Bilal PKR 200.00').
- Include a 'One-Tap Settle' button next to each suggestion that executes the payment via the existing POST /api/settle endpoint.

Option 2: Sales Tax & Discount Calculation on Shopping Checkout

Scenario:

When shopping at supermarkets (e.g., Metro or Carrefour), receipts include overall sales tax (GST %) or store discount vouchers. The checkout system must calculate net final bill amounts before splitting expenses.

Deliverables & Requirements:

1. Backend REST API Update (POST /api/shopping/checkout):

- Update payload to accept optional `tax_percent` (e.g., 5.0%) and `discount_amount` (e.g., PKR 100.00).
- Calculate total payable as: Final Amount = (Items Total * (1 + Tax%)) - Discount.
- Post the shared expense using the calculated net total and record tax/discount metadata.

2. Mobile UI Implementation:

- Update the Shopping Checkout Dialog with two new input fields: Tax (%) and Discount (PKR).
- Display live real-time net total recalculation on the checkout screen prior to user confirmation.

Option 3: Per-Category Budget Warnings & Visual Alerts

Scenario:

The household wants to prevent overspending on non-essential items (e.g., Dining out or Snacks) by setting category budget caps within the overall monthly budget.

Deliverables & Requirements:

1. Backend REST API Update (GET /api/budget/categories):

- Implement an endpoint returning total spending per category (Groceries, Utilities, Entertainment, Cleaning).
- Flag any category that accounts for more than 40% of the overall monthly household budget limit.

2. Mobile UI Implementation:

- Display a Category Spending Breakdown list on the Home Dashboard.
- Display a prominent warning badge (e.g., ⚠️ 'Groceries Limit Exceeded') if a category exceeds its threshold.

Option 4: Monthly Recurring Shared Expenses

Scenario:

Fixed bills like Internet or Maid Salary occur every month. Users want to mark an expense as recurring so it can be re-posted with one click each month.

Deliverables & Requirements:

1. Backend REST API Update:

- Add `is_recurring` boolean flag to `POST /api/expenses` schema.
- Expose `POST /api/expenses/recurring/apply` to fetch all recurring templates and auto-generate new expense records for the active month.

2. Mobile UI Implementation:

- Add a checkbox 'Mark as Monthly Recurring Bill' when logging a shared expense.
- Add a 'Generate Monthly Recurring Bills' button on the Home Dashboard.

---------- Best of Luck ----------